Análisis y Diseño de Algoritmos Avanzados

Capítulo 5 — Problema 1: Flota de drones para respuesta a desastres

Carlos Emiliano Nava Flores A00842650
Luis Enrique Castillo Carrillo A00842390

Departamento de Ciencias Computacionales

8 de noviembre de 2026

Resumen

Tras un sismo, una flota de drones lleva kits médicos desde un depósito a clínicas de campo. La ciudad se modela como un grafo dirigido sobre una rejilla $N \times N$ con zonas de exclusión aérea y viento, de modo que $c(u, v) \neq c(v, u)$ (asimetría media de 34.8 % en las matrices de costos). Se implementaron y compararon, en Python, cuatro bloques: (i) rutas con UCS, A*, IDA* (clásico e IDA*-CR) con la heurística octil×costo mínimo, cuya admisibilidad y consistencia se demuestran, y una heurística aprendida entrenada en mapas del *2D Path Planning Dataset*; (ii) carga útil con backtracking por máscaras de bits, ramificación y acotamiento (B&B) con cota fraccional y *meet-in-the-middle* (MITM), validados contra las 10 000 instancias del *Knapsack Problem Data* (0 discrepancias); (iii) asignación de plataformas y turnos como CSP con forward checking, AC-3 y MRV/LCV; y (iv) el orden de visita como TSP asimétrico, resuelto exactamente con B&B (cota MST) y Held–Karp para $K \leq 12$, y con recocido simulado, búsqueda tabú, GA con OX, ACO (MMAS), una hiperheurística UCB y dos híbridos para $K \in \{12, 25, 50, 100\}$ y $N \in \{50, 100, 150, 200\}$, con 10 semillas y el óptimo exacto (HiGHS con cortes DFJ) como referencia. Para los puntos extra se agregan un panel interactivo, la heurística aprendida con verificación empírica de la cota $w \cdot C^*$ y tres híbridos búsqueda–bioinspirados.

Índice

1 Modelo del problema **2**
1.1 La ciudad como grafo dirigido . . . 2
1.2 Heurística octil por costo mínimo . . . 2
1.3 Escenario integrado . . . 3

2 Datasets **3**

3 Rutas: UCS, A* e IDA* **4**
3.1 Algoritmos . . . 4
3.2 Validación con el dataset . . . 4
3.3 Resultados en las ciudades . . . 5
3.4 Heurística aprendida . . . 6

4 Carga útil de cada dron **8**

5 Plataformas de aterrizaje y turnos (CSP) **9**

6 Orden de visita: TSP asimétrico **12**
6.1 Métodos exactos 12
6.2 Metaheurísticas 13

7 Discusión: qué garantiza cada método **16**

8 Puntos extra **17**

9 Configuración experimental y reproducibilidad **17**

1 Modelo del problema

1.1 La ciudad como grafo dirigido

La ciudad es una rejilla $N \times N$. Cada celda libre es un vértice; las celdas bloqueadas son zonas de exclusión aérea (edificios colapsados, humo, helicópteros de rescate). Un dron puede moverse a sus 8 vecinas; no se permiten diagonales que "corten esquina" junto a una celda bloqueada. El viento es un campo suave $\mathbf{w}$: celdas $\to \mathbb{R}^2$ con $\|\mathbf{w}\| \leq 1$: un viento dominante de dirección aleatoria más una turbulencia (ruido blanco filtrado con una gaussiana). El costo de la arista $u \to v$ es

$$c(u,v) = \ell(u,v)\,(1 - \alpha\,\langle \bar{\mathbf{w}}(u,v), \hat{\mathbf{d}}(u,v)\rangle), \qquad \bar{\mathbf{w}} = \tfrac{1}{2}(\mathbf{w}(u) + \mathbf{w}(v)), \tag{1}$$

donde $\ell \in \{1, \sqrt{2}\}$ es la longitud del movimiento, $\hat{\mathbf{d}}$ su dirección unitaria y $\alpha = 0.5$. Con viento a favor el costo baja y con viento en contra sube, así que $c(u,v) - c(v,u) = -2\alpha\,\ell\,\langle \bar{\mathbf{w}}, \hat{\mathbf{d}}\rangle \neq 0$ en general. El multiplicador queda en $[1-\alpha, 1+\alpha] = [0.5, 1.5]$. Definimos el *costo mínimo por unidad*

$$m_{\min} = \min_{(u,v)\in E} \frac{c(u,v)}{\ell(u,v)} > 0,$$

que se calcula exactamente al construir el grafo. En las ciudades de prueba vale 0.582, 0.581, 0.506 y 0.513 para $N = 50$, 100, 150 y 200.

1.2 Heurística octil por costo mínimo

Sea $\mathrm{oct}(n,g) = \Delta_{\max} + (\sqrt{2}-1)\Delta_{\min}$ con $\Delta_{\max}, \Delta_{\min}$ el mayor y el menor de $|r_n - r_g|$ y $|c_n - c_g|$. La heurística pedida es

$$h(n) = m_{\min} \cdot \mathrm{oct}(n,g). \tag{2}$$

Proposición 1. *h es consistente y admisible en el grafo dirigido de* (1).

Demostración. Consistencia. La distancia octil es la distancia de camino más corto en la rejilla 8-conexa sin obstáculos, luego cumple la desigualdad del triángulo; además el movimiento $u \to v$ tiene longitud octil exactamente $\ell(u,v)$. Por tanto $\mathrm{oct}(u,g) \leq \ell(u,v) + \mathrm{oct}(v,g)$. Multiplicando por $m_{\min} > 0$ y usando $m_{\min}\,\ell(u,v) \leq c(u,v)$ (definición de $m_{\min}$): $h(u) \leq m_{\min}\ell(u,v) + h(v) \leq c(u,v) + h(v)$ para toda arista. *Admisibilidad.* Sea $n = v_0, \ldots, v_k = g$ un camino óptimo. Sumando la consistencia a lo largo del camino, $h(n) \leq \sum_i c(v_i, v_{i+1}) + h(g) = h^*(n)$ porque $h(g) = 0$. □

Observación. Los obstáculos sólo pueden alargar los caminos, así que la prueba no depende de ellos. El precio de la garantía es que h es débil cuando el viento es fuerte: con $m_{\min}$ entre 0.5 y 0.6, en los mapas de prueba el máximo de h/h^* sobre el camino óptimo fue en promedio 0.61. Eso motiva la heurística aprendida (sección 3.4).

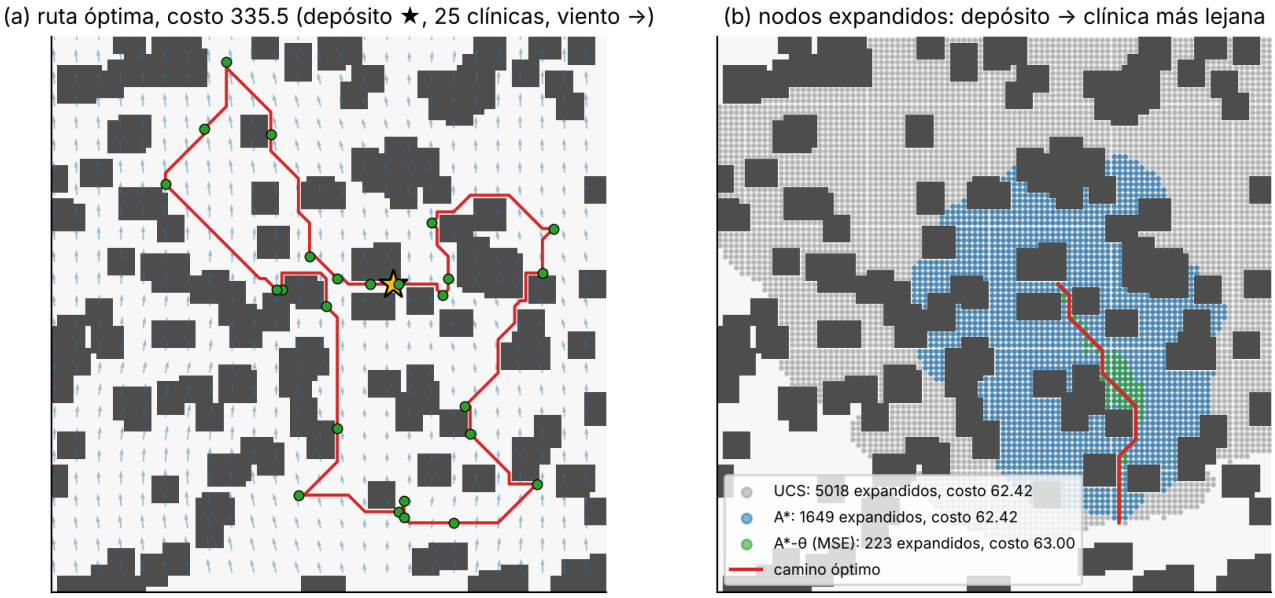


Figura 1: Misión de ejemplo ($N = 100$, $K = 25$). (a) Zonas de exclusión (gris), viento (flechas), depósito (estrella) y la ruta óptima del ATSP dibujada como los caminos de A* de cada tramo. (b) Nodos expandidos por UCS, A* y A* con la heurística aprendida en la consulta del depósito a la clínica más costosa.

1.3 Escenario integrado

El depósito y las clínicas son hospitales reales de la zona de Los Ángeles (§2): se proyectan (lat, lon) a la rejilla y cada uno se mueve a la celda libre más cercana de la componente conexa mayor, sin repetir celdas. El depósito es el LAC+USC Medical Center. Las zonas de exclusión se generan con el mismo procedimiento que los mapas del dataset de Kaggle (ruido binario y apertura morfológica, §2), a 100×100 y reescaladas a $N \times N$ para que el área bloqueada no dependa de N. La Figura 1 muestra la misión de ejemplo con $N = 100$ y $K = 25$.

2 Datasets

Los mapas son los archivos originales del *2D Path Planning Dataset* de Kaggle. Para los otros dos datasets, Kaggle no era accesible desde el entorno donde se corrieron los experimentos, así que se usaron copias públicas: una copia exacta del archivo de la mochila y una base de hospitales con la misma fuente (HIFLD). La Tabla 1 documenta cada una. El código también lee el CSV original de hospitales de Kaggle si se coloca en `datos/`.

Tabla 1: Datasets usados. Fechas de descarga: 8 de noviembre de 2026 (mapas) y 3 de noviembre de 2026 (hospitales y mochila).

Dataset (Kaggle)	Fuente usada	Preprocesamiento
2D Path Planning Dataset `kaggle.com/datasets/dcaffo/2dpathplanningdataset`	Archivo original de Kaggle (`archive.zip`, 646 MB): carpetas `train`, `validation` y `test` con un `.pt` (objeto `MapSample` de PyTorch) por mapa	Por el límite de transferencia se tomó una muestra sistemática (uno de cada k archivos del zip) de 2 506 mapas 100×100: 1 933 de `train`, 132 de `validation` y 441 de `test`. Los `.pt` se leen sin PyTorch, reconstruyendo cada tensor (`map`, `start`, `goal`, `path`) desde el zip. Celda = 1 es obstáculo; el camino de referencia es el de D* Lite. Se entrena con 300 mapas de `train` y se prueba con 100 de `test` (muestras con semilla 0).
US Hospital Locations `kaggle.com/datasets/andrewmvd/us-hospital-locations`	CovidCareMap, `github.com/covidcaremap/covid19-healthsystemcapacity` (`us_healthcare_capacity-facility-CovidCareMap.csv`); cada fila trae su `HIFLD-ID`, la misma fuente HIFLD del dataset de Kaggle	Filtro a la caja $33.70 \le \text{lat} \le 34.35$, $-118.70 \le \text{lon} \le -117.90$ (127 hospitales), sin coordenadas repetidas; proyección lineal a la rejilla y ajuste a la celda libre más cercana.
Knapsack Problem Data `kaggle.com/datasets/warcoder/knapsack-problem`	Copia exacta `knapsack_5_items.csv` en `github.com/Kralog2/Tarea3IA` (SHA-256 `55b98b9d...044c`)	10 000 instancias de 5 kits (peso, precio, capacidad, solución óptima). Se parsean los arreglos en texto; 121 filas con *Best picks* vacío (ningún kit cabe, *Best price*= 0) se toman como el vector cero. Los 50 000 kits forman el *pool* del que se muestrean instancias de $n = 10$ a 120.

3 Rutas: UCS, A* e IDA*

3.1 Algoritmos

UCS es Dijkstra con prueba de meta al expandir. **A*** usa $f = g + h$ con la heurística (2); a igual f prefiere el mayor g. Como h es consistente, ningún nodo se reabre. **IDA*** hace búsquedas en profundidad con umbral creciente sobre f. Se implementaron dos políticas de umbral:

- *IDA* clásico* (Korf): el nuevo umbral es el menor f que excedió al anterior. La primera solución es óptima, pero con costos reales casi cada iteración agrega un solo valor de f, y el número de iteraciones crece con el número de valores distintos de f.
- *IDA*-CR* (Sarkar et al., 1991): el umbral se elige entre los f excedidos para que la siguiente iteración tenga $\approx$ el doble de nodos. Si en esa iteración aparece una meta, la DFS sigue como B&B (poda $f \ge$ mejor costo) hasta agotar la iteración; entonces la solución es óptima, porque todo nodo de un camino óptimo tiene $f \le C^* \le$ umbral y se visita.

En una rejilla hay muchísimos caminos distintos hacia una misma celda (transposiciones), y sin memoria IDA* los recorre todos. Por eso ambas variantes se probaron con y sin *tabla de transposición* (TT) por iteración: se poda si se llega a un nodo con g mayor o igual al mejor g visto en esa iteración, lo cual es seguro porque el presupuesto restante sería menor o igual.

El *factor de ramificación efectivo* b^* resuelve $N_{\text{gen}} + 1 = 1 + b^* + \cdots + (b^*)^d$ (Russell y Norvig), con N_{gen} los nodos generados sin contar la raíz (la raíz es el 1 del lado derecho) y d el número de aristas de la solución; se calcula por bisección.

3.2 Validación con el dataset

Sin viento y con los costos del dataset (pasos de 1 y $\sqrt{2}$, cortes de esquina permitidos) se compararon UCS y A* con los caminos de referencia de D* Lite en los 2 506 mapas (obstáculos: 21.2 % del área).

Tabla 2: Búsqueda de rutas en las ciudades (media ± d.e. sobre las consultas resueltas; memoria pico de Python medida con `tracemalloc` en las 3 primeras consultas, para las que llegan al límite repitiendo la búsqueda hasta el mismo número de nodos; brecha respecto a C^*).

N	método	resueltas	nodos expandidos	b^*	tiempo [ms]	mem. [KiB]	brecha [%]
50	UCS	20/20	1 000 ± 563	1.391	2.6 ± 2.5	49	0.00
	A*	20/20	444 ± 385	1.277	0.7 ± 1.1	45	0.00
	IDA*-CR	10/10	297 738 ± 463 959	1.898	804.8 ± 1 235.1	156	0.00
	IDA*	3/10	217 203 ± 201 565	3.068	554.4 ± 518.8	6	0.00
	WA* (w=1.5)	20/20	259 ± 243	1.241	0.3 ± 0.3	44	0.04
	A*-θ (MSE)	20/20	157 ± 138	1.207	41.4 ± 3.5	1 552	1.06
	A*-θ (q=0.2)	20/20	151 ± 133	1.216	41.0 ± 0.5	1 552	0.02
100	UCS	20/20	2 466 ± 1 736	1.337	4.5 ± 3.2	209	0.00
	A*	20/20	949 ± 868	1.229	1.6 ± 1.5	191	0.00
	IDA*-CR	8/10	464 107 ± 1 194 830	1.886	1 293.7 ± 3 312.0	2 675	0.00
	IDA*	3/10	60 016 ± 90 031	2.895	142.5 ± 213.6	13	0.00
	WA* (w=1.5)	20/20	473 ± 522	1.191	0.7 ± 0.9	185	0.22
	A*-θ (MSE)	20/20	740 ± 1 637	1.167	115.0 ± 26.0	6 005	3.02
	A*-θ (q=0.2)	20/20	445 ± 673	1.174	119.2 ± 28.1	6 005	0.24
150	UCS	20/20	6 313 ± 3 440	1.121	11.5 ± 6.4	486	0.00
	A*	20/20	3 064 ± 2 637	1.103	5.3 ± 4.7	444	0.00
	IDA*-CR	3/10	1 211 171 ± 500 213	1.356	3 876.1 ± 1 488.0	5 014	0.00
	IDA*	0/10	–	–	–	16	–
	WA* (w=1.5)	20/20	1 920 ± 2 021	1.091	3.6 ± 4.1	430	0.17
	A*-θ (MSE)	20/20	5 927 ± 11 635	1.088	317.4 ± 49.0	13 427	2.66
	A*-θ (q=0.2)	20/20	4 487 ± 7 690	1.088	318.7 ± 56.3	13 427	0.35
200	UCS	20/20	9 372 ± 5 247	1.105	19.1 ± 11.0	914	0.00
	A*	20/20	4 766 ± 3 363	1.086	9.7 ± 7.4	820	0.00
	IDA*-CR	2/10	1 332 718 ± 1 860 151	1.501	4 196.8 ± 5 863.1	6 589	0.00
	IDA*	0/10	–	–	–	12	–
	WA* (w=1.5)	20/20	3 200 ± 2 482	1.080	6.6 ± 5.5	790	0.26
	A*-θ (MSE)	20/20	17 438 ± 21 605	1.075	479.0 ± 114.5	23 818	4.78
	A*-θ (q=0.2)	20/20	12 855 ± 26 687	1.072	470.1 ± 89.3	23 818	0.24

Los 2 506 caminos de referencia son caminos válidos de 8-vecindad, pero no son los más cortos del mapa: en mediana miden 3.1 % más que el óptimo de UCS/A*, sólo 375 coinciden con él y ninguno (0) es más corto. La razón es que el autor generó el dataset con `obst_margin` = 1 (así está en su `dataset/ds_generator.py`): su planificador no pasa por celdas vecinas a un obstáculo, salvo junto al inicio o la meta, y en efecto ningún camino de referencia lo hace. Con ese mismo modelo (obstáculos inflados una celda), el costo de A* coincide con el de referencia en 2 091 de los 2 506 mapas (83.4 %). En los 415 restantes A* encuentra un camino ligeramente más barato (mediana 0.84 %, máximo 6.0 %), y en ninguno uno más caro (0). El D* Lite publicado por el autor, corrido sobre varios de esos mapas, también da nuestro costo y no el del dataset, así que la diferencia viene de la versión del planificador con que se generó (su código intenta primero una implementación en C++ que no está en el repositorio), no de UCS/A*. UCS y A* coincidieron siempre entre sí; en promedio A* expandió 482 nodos contra 6 479 de UCS.

3.3 Resultados en las ciudades

Para cada $N \in \{50, 100, 150, 200\}$ se hicieron 20 consultas entre pares de hospitales del escenario (las mismas para todos los métodos). Las dos variantes de IDA* se corrieron en las primeras 10, con límite de 15 s y $4 \cdot 10^6$ nodos. El costo óptimo C^* se obtiene con un Dijkstra independiente (SciPy). La Tabla 2 y la Figura 2 resumen los resultados.

Lo principal:

- UCS, A* e IDA*-CR (cuando termina) son óptimos en todas las consultas. A* expande 51.8 % menos nodos que UCS en total. La ganancia es moderada porque h es conservadora ($m_{\min}$ entre

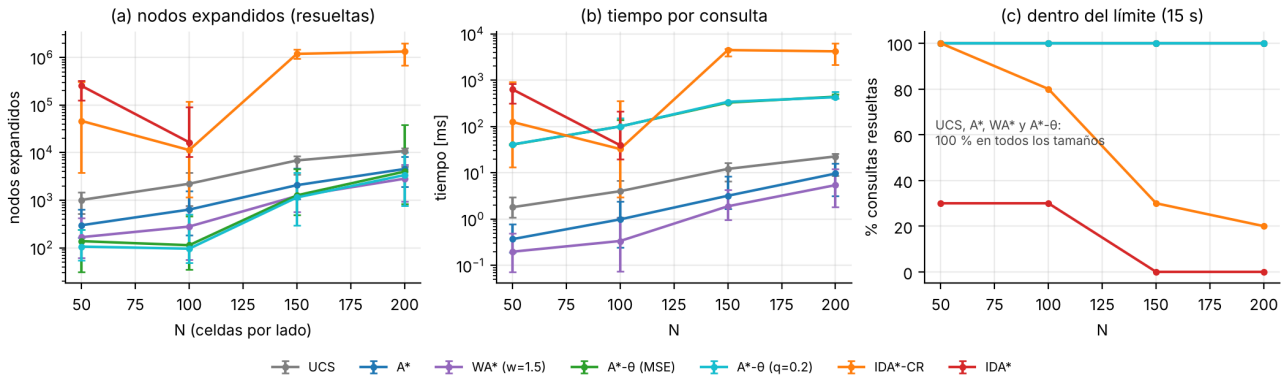


Figura 2: Escalamiento de la búsqueda de rutas con el tamaño de la rejilla (mediana y rango intercuartil de las consultas resueltas; 20 consultas por tamaño, 10 para IDA*).

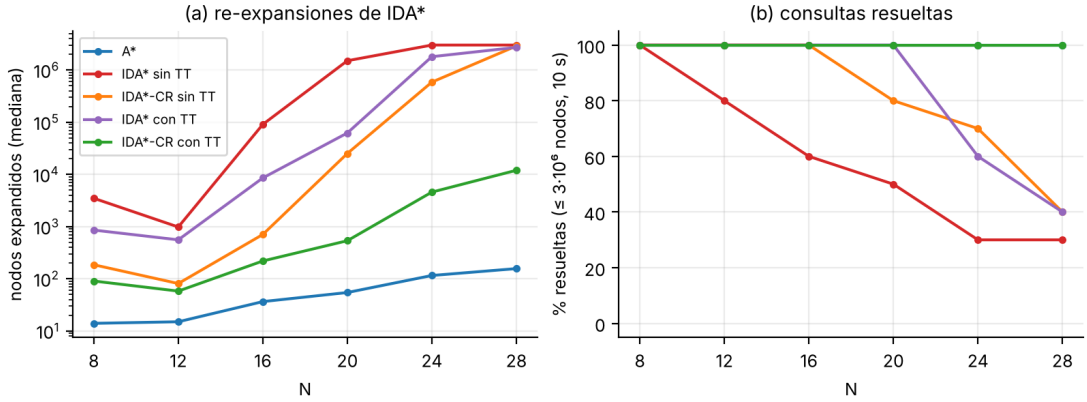


Figura 3: IDA* en rejillas pequeñas con viento (10 consultas por tamaño, límite de $3 \cdot 10^6$ nodos o 10 s). Sin tabla de transposición, el número de caminos que recorre crece exponencialmente.

0.51 y 0.58).

- El b^* de UCS y A* queda entre 1.2 y 1.4 porque d es grande (del orden de N); sirve más para comparar métodos en una misma consulta que entre tamaños. El de IDA* es mucho mayor porque cuenta todos los nodos generados en todas las iteraciones.
- IDA* resolvió 23/40 consultas con la política CR y sólo 6/40 con la clásica. Aun con TT, IDA* re-expande muchas veces los nodos a los que llega primero por un camino malo, y su tiempo es órdenes de magnitud mayor que el de A*. Su ventaja teórica (memoria $O(d)$) sólo existe sin TT, y sin TT el número de caminos explota (Figura 3): con $N = 24$ IDA* sin TT resolvió 3/10 consultas, contra 10/10 de IDA*-CR con TT, mientras A* expande una mediana de 116 nodos. En memoria, en cambio, IDA* sin TT sí cumple su promesa: con $N = 24$ su pico fue de 1.6 KiB (sólo la pila), contra 11.7 KiB de A* y 8.0 KiB de IDA*-CR con TT (medianas de 3 consultas).

3.4 Heurística aprendida

Modelo. Se aprende el cociente $r(n) = h^*(n)/\operatorname{oct}(n, g)$ y se usa $h_\theta(n) = \operatorname{oct}(n, g)\,\hat{r}(n)$, con $h_\theta(g) = 0$. Los rasgos se calculan para todas las celdas a la vez: distancias octil y euclidiana, fracción de celdas bloqueadas y número de cruces de obstáculo en 16 muestras del segmento $n \to g$, viento proyectado sobre ese segmento, densidad local de obstáculos alrededor de n y de g, $|\Delta r|$, $|\Delta c|$, densidad global y $m_{\min}$. El objetivo h^* se obtiene con un Dijkstra en el grafo inverso desde la meta.

Tabla 3: Heurística aprendida frente a A* admisible y A* ponderado ($w = 1.5$). El tiempo incluye la inferencia del modelo (119.7 ms por consulta en promedio en los mapas de prueba); la memoria (pico de Python en 3 mapas o consultas) incluye la matriz de rasgos de toda la rejilla.

método	brecha [%]	máx. [%]	óptimas [%]	expandidos	tiempo [ms]	$\bar{w}_P$	max w_P	cota ok [%]	mem. [KiB]
mapas de prueba del dataset (100×100)									
UCS	0.00	0.00	100	5 979	12.4	–	–	100	332
A*	0.00	0.00	100	3 855	9.5	–	–	100	275
WA* (w=1.5)	0.65	11.30	58	2 355	6.2	1.5	1.5	100	244
A*-θ (MSE)	1.09	10.00	3	2 835	127.0	1.126	1.671	100	6 005
A*-θ (q=0.2)	0.34	9.20	11	2 389	145.4	1.030	1.201	100	6 005
ciudades $N \in \{50, \ldots, 200\}$									
UCS	0.00	0.00	100	4 788	9.4	–	–	–	415
A*	0.00	0.00	100	2 306	4.3	–	–	–	375
WA* (w=1.5)	0.17	3.73	72	1 463	2.8	1.5	1.5	–	362
A*-θ (MSE)	2.88	21.97	15	6 065	238.2	1.424	3.532	100	11 200
A*-θ (q=0.2)	0.21	4.94	49	4 485	237.3	1.033	1.175	100	11 200

Entrenamiento. En 300 mapas de la carpeta `train` del dataset, cada uno con un viento aleatorio, y la meta del camino de referencia, se tomaron 2 000 celdas por mapa (600 000 filas). Se entrenaron dos modelos de *gradient boosting* (HistGradientBoosting de scikit-learn): con pérdida cuadrática (MSE) y con pérdida de cuantil 0.2, que sesga la predicción hacia abajo. En el 10 % de validación el error absoluto medio de $\hat{r}$ fue 0.020 y 0.029, y la fracción de celdas sobreestimadas fue 55.4 % y 20.5 %.

Garantía cuando sobreestima. h_θ no es admisible ni consistente, así que se usa A* *con reapertura* de nodos.

Proposición 2. *Sea P^* un camino óptimo de s a g y $w_P = \max\{1, \max_{n \in P^*} h(n)/h^*(n)\}$. Si A* reabre nodos cerrados cuando mejora su g y $h(g) = 0$, el costo devuelto cumple $C \le w_P \cdot C^*$.*

Demostración. Mientras g no se haya expandido, existe en OPEN un nodo $n' \in P^*$ con $g(n') = g^*(n')$: es el primer nodo de P^* que no está cerrado con su g óptimo; su predecesor en P^* sí lo está, y al expandirlo le asignó $g^*(n')$ (con reapertura, aunque n' estuviera cerrado con un g peor). Entonces $f(n') = g^*(n') + h(n') \le g^*(n') + w_P h^*(n') \le w_P\,(g^*(n') + h^*(n')) = w_P\,C^*$. Cuando A* elige g, $C = f(g) \le f(n') \le w_P C^*$. □

En cada consulta de prueba se calculó h^* en todo el mapa, w_P sobre el camino óptimo de A* y también $w_{\text{all}} = \max_n h_\theta/h^*$ sobre todas las celdas. La Tabla 3 y la Figura 4 muestran que la cota se cumplió en el 100 % (MSE) y 100 % (cuantil) de los 100 mapas de prueba y en el 100 %/100 % de las consultas en las ciudades. En los mapas de prueba el modelo MSE llegó a $w_P = 1.671$ (w_{all} hasta 2.885), con brecha media de 1.09 % y máxima de 10.00 %; el de cuantil, más conservador, tuvo $w_P \le 1.201$ y brecha media de 0.34 % (máxima 9.20 %). En las ciudades, fuera de la escala de entrenamiento, w_P llegó a 3.532 (MSE) y 1.175 (cuantil): la cota se sigue cumpliendo, pero ya no dice mucho.

En los mapas de prueba del dataset la heurística aprendida redujo los nodos expandidos respecto a A* (2 835 y 2 389 contra 3 855), pero menos de lo que su error medio sugeriría: sus errores no son suaves entre celdas vecinas, así que f sube y baja a lo largo de los caminos y A* termina explorando una banda de nodos alrededor de C^*. En las ciudades el efecto depende de la escala. Con $N = 50$ y $N = 100$, la escala de los mapas de entrenamiento, A*-θ (MSE) expandió 157 y 740 nodos contra 444 y 949 de A* (Figura 1b muestra un caso). Con $N = 200$ expandió en promedio 17 438, más que UCS (9 372), y 15 806 de esas expansiones fueron reaperturas; la mediana (4 000, contra 4 538 de A*) muestra que no falla en todas las consultas, sino que algunas explotan: las distancias salen del rango visto al entrenar con mapas de 100×100, los árboles de decisión no extrapolan y la heurística se vuelve muy inconsistente. Además, calcular h_θ en toda la rejilla cuesta 113.4 ms con $N = 100$ y 445.6 ms con $N = 200$, más que toda la búsqueda de A*.

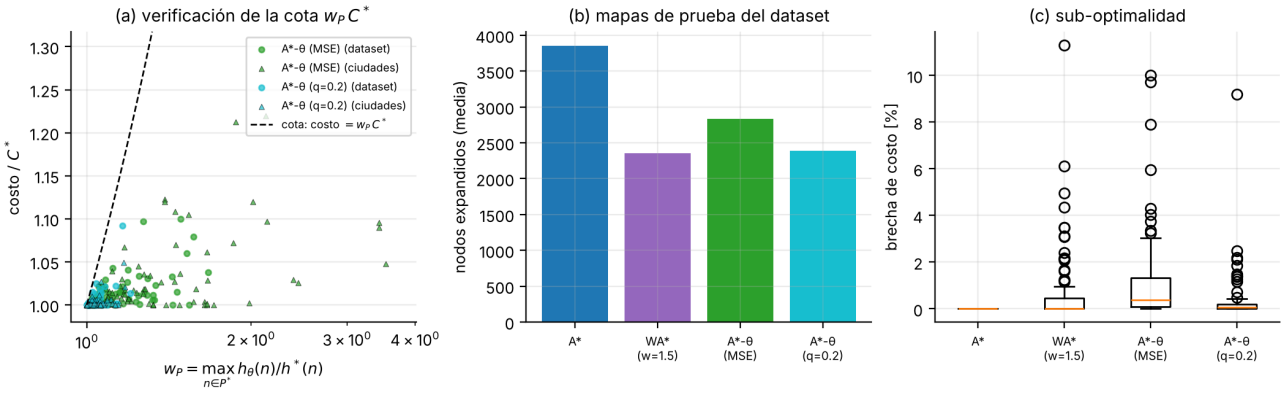


Figura 4: (a) Todas las consultas quedan bajo la diagonal costo= $w_P C^*$, como predice la Proposición 2. (b) Nodos expandidos y (c) brecha en los mapas de prueba del dataset.

A* ponderado con $w = 1.5$ expandió menos nodos en todos los tamaños con una cota *demostrable* de $1.5\,C^*$. La heurística aprendida sólo conviene si su inferencia se amortiza (muchas consultas con la misma meta), si las consultas se parecen a las de entrenamiento y si se acepta que su w es una cantidad *medida*, no garantizada.

4 Carga útil de cada dron

Cada dron elige kits $x \in \{0,1\}^n$ para maximizar el valor (prioridad médica) con capacidad W y pares incompatibles E (por ejemplo, cadena de frío y compresas térmicas):

$$\max \ \textstyle\sum_i v_i x_i \quad \text{s.a.} \quad \sum_i w_i x_i \le W, \qquad x_i + x_j \le 1 \ \ \forall (i,j) \in E.$$

Las instancias se arman con kits del dataset de Kaggle, $W = 40\,\%$ del peso total y conflictos aleatorios con grado medio 1.5; los kits se ordenan por densidad v_i/w_i.

Backtracking con máscaras de bits ($n \le 30$). El estado es (nivel i, peso, valor, máscara de elegidos, máscara `banned`). Al tomar el kit i, `banned |= conf[i]`, donde `conf[i]` es la máscara de sus incompatibles; saber si i está prohibido es un `AND` en $O(1)$. La versión "con cota" poda si el valor actual más la suma de todos los valores restantes no supera al mejor.

B&B con cota fraccional ($n \in \{30, 60, 90, 120\}$).

Proposición 3. *En un nodo con decisiones fijas para los kits $< i$, peso c_w, valor c_v y conjunto prohibido B, $UB = c_v + \mathrm{LP}(\{j \ge i : j \notin B\}, W - c_w)$, donde LP es la mochila fraccional resuelta con el voraz de Dantzig, es una cota superior del valor de cualquier completación factible.*

Demostración. Toda completación factible usa sólo kits de $\{j \ge i : j \notin B\}$, respeta la capacidad restante y además los conflictos. Quitar los conflictos y la integralidad sólo agranda la región factible, y el voraz por densidad es óptimo para la mochila fraccional. □

Se probaron DFS (incluir primero, incumbente inicial voraz) y mejor-primero por UB.

Meet-in-the-middle ($n = 40$). Se divide el conjunto en A y B empaquetando componentes conexas completas del grafo de conflictos para minimizar los conflictos cruzados. Se enumeran vectorialmente los $2^{|A|}$ y $2^{|B|}$ subconjuntos con su peso, valor, factibilidad interna y la máscara de kits de B que prohíben. Sea $\partial B \subseteq B$ el conjunto de kits de B con algún conflicto en A; para cada patrón $\beta \subseteq \partial B$ se combinan los subconjuntos de B cuya intersección con ∂B es β con los de A compatibles con β, mediante el MITM clásico: B ordenado por peso con máximo prefijo de valor y búsqueda binaria de $W - w(A)$. Es exacto, cuesta $O(2^{|\partial B|}\, 2^{n/2}\, n)$ y usa $O(2^{n/2})$ memoria. Con valor = peso resuelve "cargar lo más cerca posible de W". En este experimento el peso de cada kit es $1000\, w_i + v_i$ (gramos) para que llegar exactamente a W no sea trivial.

Validación con Kaggle. En las 10 000 instancias, backtracking, B&B (DFS y mejor-primero), MITM y una DP de control reprodujeron *Best price* sin ninguna discrepancia (0), y todos los *Best picks* del dataset son factibles y alcanzan ese valor. Tiempo medio por instancia: 31 μs (BT), 34 μs (B&B), 179 μs (MITM). Con 5 kits la memoria es mínima: el pico por instancia (el máximo en 200 instancias) fue de 9.1 KiB (BT), 8.4 KiB (B&B DFS), 8.4 KiB (mejor-primero), 9.0 KiB (MITM) y 2.8 KiB (DP).

Resultados. Cuando terminan, todos los métodos exactos coinciden con HiGHS. Sin cota el backtracking recorre casi los 2^{n+1} nodos (373 896 con $n = 22$); con la cota trivial baja a 6 185 y con $n = 30$ visita 134 936 en promedio. El B&B con cota fraccional necesita órdenes de magnitud menos nodos y resuelve casi todas las instancias en milisegundos, pero su costo depende mucho de la instancia: con $n = 120$ hubo una en la que la DFS llegó al límite de 60 s (11 755 520 nodos) y la versión mejor-primero agotó su límite de memoria de la cola. La DFS se detuvo con una incumbente de valor 830, que es la óptima (830) pero sin certificado; la de mejor-primero se quedó en 817, con brecha de 1.57 %, porque explora por cota y no profundiza hasta encontrar buenas soluciones completas. La razón de fondo es que la cota fraccional ignora los conflictos: si varios kits de alta densidad son incompatibles entre sí, la cota queda muy por encima del óptimo y casi no poda. HiGHS, cuya relajación lineal sí incluye $x_i + x_j \leq 1$, resolvió esa instancia en 65 ms. Mejor-primero expande menos nodos que DFS, pero guarda toda la frontera: hasta 68.8 MiB contra 15 KiB de la DFS. (La memoria de HiGHS no aparece: corre en C++, fuera de lo que mide `tracemalloc`.) El GA (base bioinspirada, 10 semillas, $60n$ evaluaciones) quedó a 0.34 % del óptimo en promedio y lo encontró en el 55 % de las corridas. Para cargar cerca de W con $n = 40$, MITM tardó 100 ms en promedio, usó 74.0 MiB (las listas de 2^{20} subconjuntos de cada mitad, contra 10 KiB del B&B) y dejó una holgura media de 320.2 gramos, mientras que B&B, cuya cota fraccional no poda cuando valor = peso, llegó al límite de 5 s en 7/10 instancias. Este problema (suma de subconjuntos con pesos grandes) también es difícil para HiGHS, que con 20 s de límite y tolerancia de optimalidad 0 certificó el óptimo en 30/60 instancias. (Con su tolerancia relativa por omisión, 10^{-4}, HiGHS daba por "óptimas" cargas hasta 26 g menores que la de MITM: un certificado de MILP vale sólo hasta su tolerancia.) En las 44/60 instancias con algún certificado independiente (HiGHS o B&B terminado), MITM coincidió con él, y en ninguna instancia HiGHS encontró una carga mejor que la de MITM.

5 Plataformas de aterrizaje y turnos (CSP)

Modelo. Variables: una por entrega d. Dominio: $D_d = \{(p, t) : p \in \text{plataformas permitidas},\ t \in [e_d, l_d]\}$, donde $[e_d, l_d]$ es la ventana de llegada del dron y los drones pesados sólo pueden usar las plataformas reforzadas (primera y última fila). Para todo par $d \neq d'$ con valores (p, t) y (p', t'):

$$t \neq t' \quad \vee \quad p' \notin N[p], \qquad N[p] = \{p\} \cup \text{vecinos}(p),$$

es decir, un dron por plataforma por turno y nunca dos plataformas adyacentes a la vez. Los dominios son *bitsets*: (p, t) es el bit $tP + p$ y conf$[a]$ es la máscara de los valores incompatibles con a, de modo

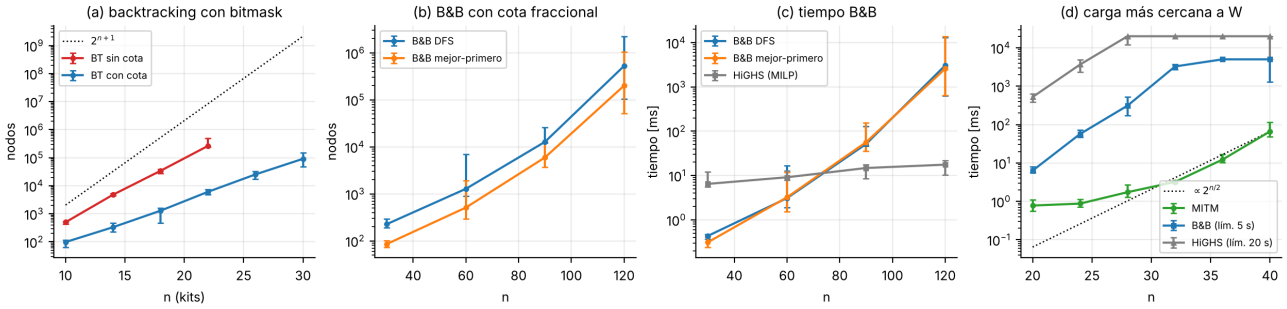


Figura 5: Carga útil (mediana y rango intercuartil de 10 instancias por n): (a) backtracking con y sin cota; (b,c) B&B con cota fraccional; (d) MITM frente a B&B y HiGHS para cargar lo más cerca posible de W.

Tabla 4: B&B con cota fraccional, HiGHS y GA (10 instancias por n; GA con 10 semillas por instancia). "Termina": corridas que demostraron optimalidad dentro de 60 s.

n	método	nodos	tiempo [ms]	mem. [KiB]	brecha [%]	óptimo [%]	termina
30	BT con cota	86 115 ± 87 883	22.12 ± 21.15	9	0.000 ± 0.000	100	–
	B&B DFS	239 ± 74	0.45 ± 0.11	9	0.000 ± 0.000	100	10/10
	B&B mejor-primero	88 ± 33	0.32 ± 0.12	9	0.000 ± 0.000	100	10/10
	HiGHS (MILP)	–	30.51 ± 71.70	–	0.000 ± 0.000	100	–
	GA (base bioinspirada)	–	75.72 ± 19.31	22	0.046 ± 0.230	95	–
60	B&B DFS	5 167 ± 6 918	12.41 ± 16.74	10	0.000 ± 0.000	100	10/10
	B&B mejor-primero	1 887 ± 2 708	11.59 ± 17.06	21	0.000 ± 0.000	100	10/10
	HiGHS (MILP)	–	10.37 ± 3.30	–	0.000 ± 0.000	100	–
	GA (base bioinspirada)	–	218.12 ± 28.88	28	0.387 ± 0.585	51	–
90	B&B DFS	26 466 ± 33 395	147.72 ± 242.66	11	0.000 ± 0.000	100	10/10
	B&B mejor-primero	12 213 ± 18 029	160.96 ± 276.09	491	0.000 ± 0.000	100	10/10
	HiGHS (MILP)	–	16.40 ± 11.40	–	0.000 ± 0.000	100	–
	GA (base bioinspirada)	–	502.97 ± 111.63	34	0.354 ± 0.736	49	–
120	B&B DFS	1 983 490 ± 3 608 546	10 641.29 ± 18 428.13	15	0.000 ± 0.000	100	9/10
	B&B mejor-primero	560 034 ± 700 984	7 783.98 ± 9 999.03	28 359	0.157 ± 0.495	90	9/10
	HiGHS (MILP)	–	21.88 ± 17.52	–	0.000 ± 0.000	100	–
	GA (base bioinspirada)	–	740.44 ± 103.92	41	0.573 ± 0.628	24	–

que forward checking es `dom[y] &= ~conf[a]`.

Algoritmos. Backtracking cronológico (BT); forward checking (FC); AC-3, como preprocesamiento o mantenido en cada nodo (MAC), con *revise*(X_i, X_j): a tiene soporte sii $D_j \,\&\, \neg\text{conf}[a] \neq 0$. En MAC, después de forward checking la cola de AC-3 empieza con los arcos (X_k, X_j) hacia cada variable X_j cuyo dominio se redujo, para que esas reducciones se propaguen entre las variables que faltan. MRV elige la variable con menos valores *legales* restantes (en BT, los del dominio que no chocan con lo ya asignado), con desempate por grado (entregas con ventana traslapada), y LCV prueba primero el valor que elimina menos valores ajenos. Min-conflicts (10 semillas) sirve de referencia de búsqueda local. Todas las soluciones se revisaron con un verificador independiente (100 % válidas de 1126).

Resultados. Con un hub de 3×4 plataformas, adyacencia de 8 y $T = 8$ turnos caben a lo sumo 4 drones por turno (32 en total). La Tabla 6 y la Figura 6 muestran la transición de fase: lejos de la capacidad todos resuelven; cerca de ella BT y FC con orden estático se atoran, MRV es lo que más ayuda y LCV ayuda en las instancias más difíciles. AC-3 sólo como preprocesamiento no cambia nada: en la raíz todos los valores tienen soporte, porque la restricción es "suelta" (un valor (p, t) pierde soporte en X_j sólo si todo D_j cae en el mismo turno y en la vecindad de p). Mantenido en cada nodo (MAC),

Tabla 5: MITM para cargar lo más cerca posible de W (10 instancias por n).

n	MITM [ms]	subconjuntos	$\lvert\partial B\rvert$	$W-$carga	consistente [%]	mem. MITM [KiB]	B&B [ms]	B&B al límite	mem. B&B [KiB]
20	1.0 ± 0.6	698	2.3 ± 2.8	336.4	100	79	8 ± 4	0/10	9
24	1.1 ± 0.6	2 077	2.3 ± 2.6	388.0	100	303	73 ± 57	0/10	9
28	2.2 ± 1.2	7 584	3.2 ± 2.9	463.8	100	1 188	362 ± 213	0/10	9
32	3.3 ± 0.4	17 914	0.6 ± 1.1	364.1	100	4 740	3 069 ± 1 341	1/10	9
36	15.0 ± 7.0	63 467	1.8 ± 2.9	351.3	100	18 949	4 503 ± 1 575	9/10	9
40	100.5 ± 91.6	260 230	3.4 ± 3.8	320.2	100	75 781	3 505 ± 2 408	7/10	10

Tabla 6: CSP con hub de 3×4 y $T = 8$ (10 instancias por D). Por método: % de instancias terminadas dentro de 10^5 nodos o 5 s (mediana de nodos); min-conflicts: % resueltas.

D	sat/unsat /?	BT	BT+MRV	FC	FC+MRV	FC+MRV +LCV	AC-3+FC +MRV+LCV	MAC+MRV +LCV	Min-conflicts
12	10/0/0	100 (35)	100 (36)	100 (12)	100 (12)	100 (12)	100 (12)	100 (12)	100
16	10/0/0	90 (55)	100 (53)	100 (16)	100 (16)	100 (16)	100 (16)	100 (16)	100
20	10/0/0	80 (102)	100 (93)	90 (21)	100 (20)	100 (20)	100 (20)	100 (20)	100
22	10/0/0	80 (96)	100 (98)	80 (22)	100 (22)	100 (22)	100 (22)	100 (22)	100
24	10/0/0	40 (100 002)	100 (157)	80 (33)	100 (24)	100 (24)	100 (24)	100 (24)	100
26	9/0/1	30 (100 000)	80 (186)	40 (100 000)	90 (26)	90 (26)	90 (26)	90 (26)	90
28	3/3/4	0 (100 009)	50 (52 583)	0 (100 000)	50 (50 216)	60 (422)	60 (422)	60 (196)	30
30	6/1/3	0 (100 010)	20 (100 009)	10 (100 000)	30 (100 000)	60 (16 243)	60 (16 243)	50 (3 599)	67
32	1/1/8	0 (100 006)	10 (100 012)	0 (100 000)	10 (100 000)	20 (100 000)	20 (100 000)	20 (24 192)	18

en cambio, sí poda más que forward checking cuando los dominios ya están casi vacíos, que es justo la región difícil: la mediana de nodos baja de 422 a 196 con $D = 28$, de 16 243 a 3 599 con $D = 30$ y de 100 000 (el límite) a 24 192 con $D = 32$, comparado con FC+MRV+LCV. Cada nodo de MAC cuesta mucho más, porque AC-3 revisa hasta $O(D^2)$ arcos (2 061 ms contra 236 ms en promedio con $D = 30$), así que con el límite de 5 s las dos variantes terminan proporciones parecidas de instancias (5/10 y 6/10 con $D = 30$). Ninguno de los métodos completos prueba rápido la insatisfacibilidad de las instancias saturadas (son problemas tipo palomar), y min-conflicts nunca puede probarla.

Tabla 7: Escalamiento con un hub de 4×5, adyacencia de 4 y $T = 12$: % terminadas / tiempo medio [ms].

D	BT+MRV	FC+MRV	FC+MRV+LCV	MAC+MRV+LCV	Min-conflicts
20	100 / 0.3	100 / 0.3	100 / 1.3	100 / 13.0	100 / 0.5
40	100 / 0.5	100 / 0.6	100 / 2.9	100 / 62.7	100 / 1.0
60	100 / 0.9	100 / 1.0	100 / 6.3	100 / 210.1	100 / 2.2
80	100 / 1.4	100 / 1.8	100 / 9.2	100 / 452.9	100 / 3.7
100	80 / 10.2	80 / 163.3	100 / 12.0	100 / 858.9	100 / 21.1

En el hub grande todas las instancias resultaron satisfacibles y FC+MRV+LCV y MAC+MRV+LCV las resuelven sin retroceder (exactamente D nodos), así que la propagación extra de MAC no tiene nada que ahorrar y sólo cuesta: con $D = 100$, FC+MRV+LCV tardó 12.0 ms y MAC+MRV+LCV 858.9 ms en promedio, con memoria pico de 211 y 1 155 KiB (Tabla 8). Con bitsets, guardar una copia de los dominios por nivel es barato; lo que ocupa memoria en MAC es la cola de AC-3. BT+MRV usa la menor memoria (72 KiB) porque no copia dominios, pero con $D = 100$ ya no terminó todas las instancias. Min-conflicts resolvió todas usando 84 KiB.

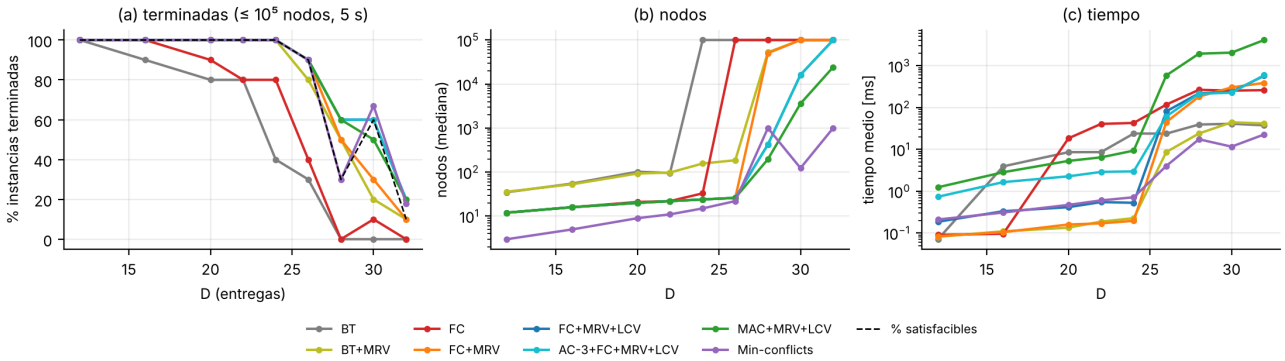


Figura 6: Transición de fase del CSP de plataformas y turnos.

Tabla 8: Memoria pico de Python [KiB] en el hub de 4×5 (instancia 0 de cada D; min-conflicts con la semilla 0).

método	$D = 20$	$D = 40$	$D = 60$	$D = 80$	$D = 100$
BT+MRV	18	28	43	62	72
FC+MRV	24	51	94	149	212
FC+MRV+LCV	24	50	93	150	211
MAC+MRV+LCV	45	175	275	930	1 155
Min-conflicts	46	54	66	76	84

6 Orden de visita: TSP asimétrico

La matriz $C \in \mathbb{R}^{(K+1)\times(K+1)}$ se obtiene con Dijkstra dirigido desde el depósito y cada clínica; por el viento $C \neq C^{\top}$. Un recorrido es una permutación de las clínicas que sale y regresa al depósito.

6.1 Métodos exactos

Proposición 4. *Para un prefijo $0 \to \cdots \to u$ con costo g y conjunto restante R, con $s_{ij} = \min(c_{ij}, c_{ji})$,*

$$LB = \max\Big\{ g + \mathrm{MST}_s(R \cup \{u, 0\}),\quad g + \mathrm{MST}_s(R) + \min_{r \in R} c_{ur} + \min_{r \in R} c_{r0} \Big\}$$

es una cota inferior del costo de cualquier recorrido que extienda el prefijo.

Demostración. El resto del recorrido es un camino hamiltoniano $u \to R \to 0$. Ignorando direcciones es un árbol de expansión de $R \cup \{u, 0\}$, y cada arista dirigida cuesta al menos su s_{ij}; eso da la primera cota. Quitando las dos aristas extremas queda un camino que es árbol de expansión de R, y esas dos aristas cuestan al menos $\min_r c_{ur}$ y $\min_r c_{r0}$. □

El B&B es en profundidad, ordena los hijos por c_{ur} y poda con $LB \geq$ mejor. Se comparó sin cota inicial, con la cota del vecino más cercano (VMC) y con la de ACO+LS (híbrido exacto + bioinspirado). Held–Karp ($O(2^K K^2)$) y la fuerza bruta ($K \leq 8$) sirven de control. Para $K > 12$ el óptimo se obtuvo con HiGHS: formulación de asignación con cortes de 2-ciclos y cortes de subciclos de Dantzig–Fulkerson–Johnson agregados de forma perezosa (a lo sumo 324.2 s por instancia). Como HiGHS declara óptimo dentro de una tolerancia relativa (10^{-4} por omisión), las 16 instancias se resolvieron otra vez con tolerancia 0 y los óptimos fueron idénticos; además, ninguna de las 1 120 corridas de metaheurísticas encontró un recorrido más barato. La relajación de asignación (AP) queda en promedio 26.5 % por debajo del óptimo.

Tabla 9: ATSP exacto con $K \leq 12$ ($N = 100$, 10 subconjuntos de clínicas por K). Para Held–Karp se reportan los estados $2^K K$; el tiempo de "+ACO+LS" incluye 2 000 evaluaciones de ACO+LS.

K	método	nodos / estados	tiempo [ms]	mem. [KiB]	óptimo [%]
6	Fuerza bruta	720 ± 0	3.4 ± 1.1	3.8	100
	Held–Karp (DP)	384 ± 0	1.2 ± 0.3	10.8	100
	B&B (MST)	73 ± 31	1.1 ± 0.5	7.5	100
	B&B (MST) + VMC	71 ± 31	1.0 ± 0.5	7.5	100
	B&B (MST) + ACO+LS	57 ± 28	33.7 ± 6.3	7.3	100
8	Fuerza bruta	40 320 ± 0	185.2 ± 27.1	3.9	100
	Held–Karp (DP)	2 048 ± 0	5.4 ± 0.3	38.3	100
	B&B (MST)	340 ± 323	8.2 ± 6.2	10.4	100
	B&B (MST) + VMC	338 ± 323	7.5 ± 5.6	10.4	100
	B&B (MST) + ACO+LS	303 ± 294	44.6 ± 9.0	10.3	100
10	Held–Karp (DP)	10 240 ± 0	27.0 ± 0.5	207.9	100
	B&B (MST)	1 695 ± 1 238	61.3 ± 38.0	14.0	100
	B&B (MST) + VMC	1 693 ± 1 238	60.5 ± 37.6	14.0	100
	B&B (MST) + ACO+LS	1 542 ± 1 200	96.1 ± 36.4	13.9	100
12	Held–Karp (DP)	49 152 ± 0	139.8 ± 24.8	983.6	100
	B&B (MST)	67 492 ± 107 107	2 568.7 ± 3 718.8	16.4	100
	B&B (MST) + VMC	67 490 ± 107 107	2 568.4 ± 3 758.9	18.1	100
	B&B (MST) + ACO+LS	63 223 ± 99 126	2 375.7 ± 3 334.8	18.1	100

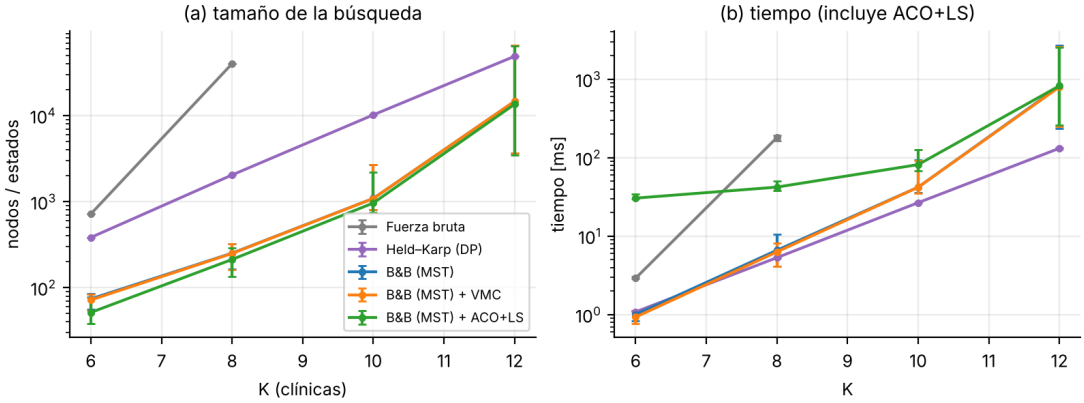


Figura 7: ATSP exacto con $K \leq 12$: nodos (estados para Held–Karp, permutaciones para fuerza bruta) y tiempo; mediana y rango intercuartil de 10 instancias por K.

Con $K = 12$ el B&B con cota MST expandió 67 492 nodos en promedio (2 569 ms, 16.4 KiB de memoria pico, frente a 983.6 KiB de la tabla de Held–Karp); con la cota inicial de ACO+LS bajó a 63 223 (2 376 ms incluyendo ACO), y con la del vecino más cercano casi no cambió. Held–Karp tardó 140 ms. Todos los métodos exactos coincidieron (100 %). Con este tamaño la DP gana: $2^K K^2$ es pequeño, mientras que la cota MST pierde fuerza con la asimetría (usa $\min(c_{ij}, c_{ji})$ e ignora la dirección). La cota inicial ayuda poco porque el B&B, al visitar primero los hijos más baratos, encuentra pronto incumbentes casi óptimas; lo que domina es demostrar que no hay nada mejor.

6.2 Metaheurísticas

Todas empiezan de permutaciones aleatorias y tienen el mismo presupuesto de $3 \cdot 10^4$ evaluaciones de la función objetivo (cada recorrido construido o vecino evaluado cuenta una vez).

- **SA**: vecino aleatorio entre swap, inserción y 2-opt (que en el ATSP invierte el sentido del segmento); T_0 calibrada para aceptar la mitad de los empeoramientos y enfriamiento geométrico hasta $10^{-3} T_0$.

Tabla 10: Brecha contra el óptimo exacto [%] (media ± d.e. de 10 semillas) para las 16 combinaciones (N, K). En negritas el mejor promedio de cada fila.

K	N	óptimo	SA	Tabu	GA-OX	ACO (MMAS)	UCB-HH	GA memético	ACO+LS
12	50	131.6	**0.00 ± 0.00**	**0.00 ± 0.00**	0.08 ± 0.18	0.42 ± 0.00	**0.00 ± 0.00**	0.13 ± 0.20	0.42 ± 0.00
12	100	276.6	**0.00 ± 0.00**	**0.00 ± 0.00**	0.19 ± 0.25	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.00 ± 0.00**
12	150	453.4	**0.00 ± 0.00**	**0.00 ± 0.00**	0.20 ± 0.43	0.28 ± 0.00	**0.00 ± 0.00**	0.13 ± 0.33	0.28 ± 0.00
12	200	582.3	**0.00 ± 0.00**	**0.00 ± 0.00**	0.07 ± 0.06	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.00 ± 0.00**
25	50	200.3	0.32 ± 0.42	**0.08 ± 0.26**	3.25 ± 2.19	0.76 ± 0.30	0.80 ± 1.43	1.77 ± 2.63	0.24 ± 0.39
25	100	335.5	0.45 ± 1.35	0.02 ± 0.06	2.41 ± 1.99	0.19 ± 0.27	**0.00 ± 0.00**	1.09 ± 1.53	0.02 ± 0.06
25	150	575.9	0.28 ± 0.40	0.27 ± 0.42	1.85 ± 1.77	1.59 ± 1.28	**0.23 ± 0.41**	0.60 ± 0.48	0.73 ± 0.56
25	200	856.4	**0.00 ± 0.00**	**0.00 ± 0.00**	0.05 ± 0.15	0.85 ± 1.64	**0.00 ± 0.00**	**0.00 ± 0.00**	0.05 ± 0.15
50	50	258.4	2.10 ± 1.93	3.82 ± 1.94	6.88 ± 3.15	2.02 ± 1.72	1.48 ± 1.27	4.51 ± 2.54	**0.21 ± 0.41**
50	100	505.6	2.50 ± 3.32	4.71 ± 3.94	11.38 ± 4.54	4.75 ± 1.98	1.03 ± 2.17	4.88 ± 3.57	**0.36 ± 0.68**
50	150	827.9	2.73 ± 2.67	5.58 ± 2.29	6.18 ± 4.48	1.06 ± 1.46	2.06 ± 1.56	5.31 ± 5.21	**0.80 ± 0.40**
50	200	1 048.8	2.59 ± 2.96	2.68 ± 2.73	8.30 ± 6.01	1.93 ± 1.02	1.91 ± 3.29	3.43 ± 3.24	**0.42 ± 0.41**
100	50	342.5	9.38 ± 3.54	9.37 ± 3.51	76.81 ± 10.09	10.05 ± 2.51	**8.21 ± 1.89**	21.87 ± 5.91	8.26 ± 2.18
100	100	682.5	16.05 ± 3.38	12.59 ± 4.43	96.12 ± 12.29	11.36 ± 1.82	13.53 ± 2.72	24.41 ± 6.09	**10.29 ± 2.06**
100	150	1 121.4	10.76 ± 3.59	13.53 ± 4.26	85.48 ± 9.61	**7.50 ± 0.75**	11.09 ± 2.82	23.50 ± 5.01	7.60 ± 2.49
100	200	1 481.3	11.67 ± 5.25	14.75 ± 5.44	96.18 ± 12.18	6.91 ± 2.28	8.62 ± 4.54	19.79 ± 5.68	**5.45 ± 1.47**

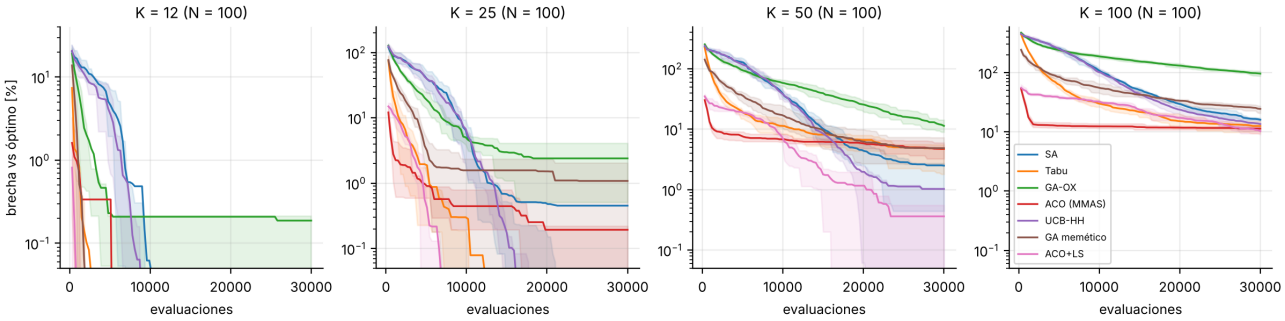


Figura 8: Convergencia con $N = 100$: brecha contra el óptimo (línea: media de 10 semillas; banda: rango intercuartil). Las brechas nulas se dibujan en 10^{-3} por la escala logarítmica.

- **Tabú**: lista de candidatos de 30 vecinos por iteración (70 % inserciones granulares de una ciudad junto a uno de sus 10 predecesores más baratos, como en la búsqueda tabú granular de Toth y Vigo, y 30 % swap/2-opt); atributo tabú: las ciudades movidas, con permanencia $\approx K/4$; aspiración por mejora global.
- **GA-OX**: población 60, torneo de 3, cruza OX (Davis), mutación por inversión ($p = 0.5$) y elitismo de 2.
- **ACO (MMAS)**: 20 hormigas, $\alpha = 1$, $\beta = 3$, $\rho = 0.1$, límites $\tau_{\min}, \tau_{\max}$ de Stützle y Hoos; depósito alternado entre la mejor de la iteración y la global; construcción vectorizada.
- **UCB-HH**: hiperheurística de selección; UCB1 elige el operador (swap, inserción o 2-opt) con recompensa 1 si mejora la solución actual; aceptación de SA con la misma temperatura.
- **Híbridos** (puntos extra): GA memético (población 30 y búsqueda local granular de $2K$ evaluaciones en el 2 % de los hijos) y ACO+LS (búsqueda local sobre la mejor hormiga de cada iteración); además el B&B con cota inicial de ACO+LS de la sección anterior.

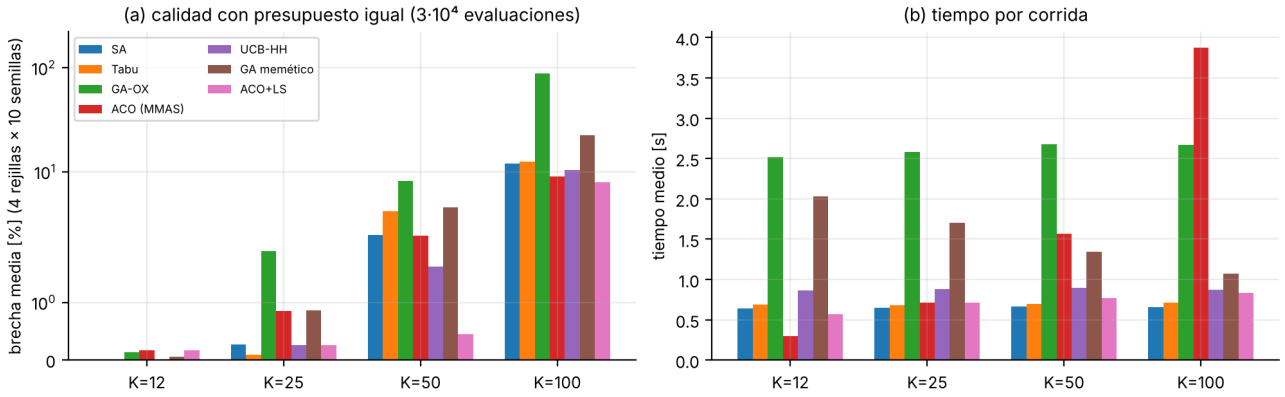


Figura 9: Brecha media y tiempo por corrida, promediados sobre las cuatro rejillas.

Tabla 11: Tiempo medio por corrida [s] / memoria pico de Python [KiB]. La memoria se midió con la semilla 0 y 5 000 evaluaciones: el pico lo fijan la población o la matriz de feromona, no el número de evaluaciones.

K	SA	Tabu	GA-OX	ACO (MMAS)	UCB-HH	GA memético	ACO+LS
12	0.64 / 12	0.69 / 14	2.52 / 41	0.30 / 37	0.86 / 31	2.03 / 32	0.57 / 37
25	0.65 / 16	0.68 / 23	2.58 / 59	0.71 / 68	0.88 / 31	1.71 / 43	0.71 / 72
50	0.67 / 18	0.70 / 49	2.68 / 101	1.57 / 186	0.89 / 34	1.34 / 75	0.77 / 205
100	0.66 / 21	0.71 / 169	2.67 / 195	3.88 / 613	0.87 / 36	1.07 / 181	0.83 / 691

Resultados. Con $K = 12$ todos los métodos quedan a menos de 0.2 % del óptimo, y SA, Tabú y UCB-HH lo encuentran en las 40 corridas. La diferencia aparece al crecer K: el mejor promedio fue Tabu con $K = 25$ (0.09 %), ACO+LS con $K = 50$ (0.45 %) y ACO+LS con $K = 100$ (7.90 %). Con $K = 100$ las brechas medias fueron: SA 11.97, Tabú 12.56, GA-OX 88.65, ACO 8.96, UCB-HH 10.36, memético 22.39 y ACO+LS 7.90 %. Algunas conclusiones:

- El GA puro es el más débil con presupuesto fijo: OX conserva el orden relativo, pero recombinar sin búsqueda local avanza muy despacio en un espacio de $K!$ permutaciones. Agregarle búsqueda local (memético) reduce su brecha a la cuarta parte con $K = 100$.
- ACO+LS fue el más robusto para $K \geq 50$: la feromona concentra la construcción en aristas buenas y la búsqueda local granular corrige el detalle. ACO sin búsqueda local también escala bien en calidad, pero es el más lento con $K = 100$ porque cada construcción cuesta $O(K^2)$.
- UCB-HH superó a SA, que usa la misma aceptación y temperatura pero elige el operador al azar, en $K = 50$ y $K = 100$. En promedio UCB eligió 2-opt el 47.0 %, inserción el 29.4 % y swap el 23.6 % de las veces (Figura 10): aprendió que invertir segmentos mejora más seguido.
- El tamaño de la rejilla N no muestra una tendencia clara: sólo cambia la matriz C, y la brecha depende sobre todo de K.
- Con $K = 100$ las curvas de la Figura 8 siguen bajando al final del presupuesto: con más evaluaciones todos mejorarían, y el orden entre métodos se mantiene a lo largo de la curva.

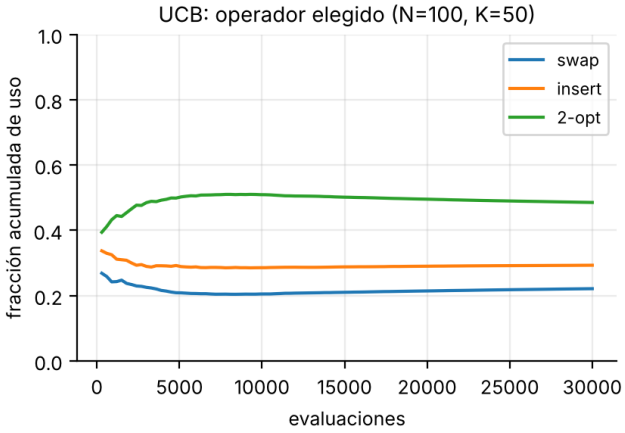


Figura 10: Fracción acumulada de uso de cada operador en la hiperheurística UCB ($N = 100$, $K = 50$, media de 10 semillas).

7 Discusión: qué garantiza cada método

Tabla 12: Garantías de los métodos usados.

Método	Garantía
UCS, A* (h consistente), IDA*/IDA*-CR	**Exactos**: completos y óptimos (costos > 0). A* no reabre nodos y es óptimamente eficiente para su h; IDA* usa $O(d)$ memoria sin TT.
A* ponderado (w)	Completo, costo $\leq w\,C^*$ demostrable.
A* con h_θ aprendida	Completo (grafo finito, con reapertura). Pierde la optimalidad, la consistencia (hay reaperturas) y la cota a priori: $C \leq w_P C^*$ sólo vale para el w_P *medido*, que no se conoce antes de resolver y puede crecer fuera de la distribución de entrenamiento.
Backtracking, B&B, MITM, Held–Karp, B&B-MST, HiGHS	**Exactos**.
BT, FC, MAC (CSP)	**Completos**: deciden satisfacible/insatisfacible (si terminan).
Min-conflicts	Incompleto: no puede demostrar insatisfacibilidad.
SA	**Probabilísticamente completo** sólo con enfriamiento logarítmico (Hajek, 1988); con el enfriamiento geométrico usado no hay garantía.
GA elitista con mutación	Converge en probabilidad al óptimo (Rudolph, 1994): la inversión genera cualquier permutación con probabilidad positiva y el elitismo no pierde lo encontrado.
ACO (MMAS)	$\tau_{\min} > 0$ da convergencia en valor: la probabilidad de haber construido el óptimo tiende a 1 (Stützle y Dorigo, 2002).
Tabú, UCB-HH	Sin garantía de optimalidad. La cota de regret de UCB1 supone recompensas estacionarias, y aquí la utilidad de cada operador cambia durante la búsqueda.

Memoria. Las diferencias de memoria siguen a las estructuras de datos. A* y UCS guardan g, padres y frontera para toda la rejilla ($O(V)$; con $N = 200$, 820 y 914 KiB en promedio). IDA* sin tabla de transposición sólo guarda la pila ($O(d)$), pero en rejillas eso lo vuelve impráctico; con la tabla vuelve a $O(V)$ (6 589 KiB para IDA*-CR con $N = 200$). En la mochila, la DFS guarda una rama ($O(n)$) y mejor-primero toda la frontera (hasta 68.8 MiB), y MITM sus $2^{n/2}$ subconjuntos por mitad (74.0 MiB con $n = 40$). Held–Karp necesita su tabla de $2^K K$ estados, MAC una cola de hasta $O(D^2)$ arcos,

ACO la matriz de feromona $O(K^2)$ (613 KiB con $K = 100$) y SA, Tabú y UCB-HH sólo un par de permutaciones (21 y 169 KiB).

En resumen, para las decisiones que sí admiten demostración en el tiempo disponible conviene usar los métodos exactos con buenas cotas: A* con la heurística consistente para las rutas (milisegundos en rejillas de 200×200), B&B con cota fraccional para la carga (casi siempre en milisegundos con $n \leq 120$, aunque con instancias difíciles cuando los kits densos son incompatibles) y Held–Karp para $K \leq 12$. Cuando el problema crece, las metaheurísticas híbridas dan buenas soluciones en menos de un segundo (ACO+LS: 0.45 % con $K = 50$ y 7.90 % con $K = 100$), pero sólo un método exacto (aquí HiGHS con cortes, hasta 324.2 s) puede certificar la brecha. La heurística aprendida muestra el intercambio central: renuncia a las garantías a cambio de información; en la escala de entrenamiento ahorra nodos, fuera de ella deja de ayudar, y A* ponderado da un ahorro parecido con una cota demostrable.

8 Puntos extra

- **Panel interactivo** (`dashboard/panel_drones.html`, un solo archivo sin dependencias): mapa de la misión con capas de viento, rutas de cada método y nodos expandidos; curvas de convergencia para las 16 combinaciones (N, K); tablas de búsqueda, mochila y CSP; y el horario de plataformas.
- **Heurística aprendida** con dos pérdidas, verificación empírica de la Proposición 2 y su prueba.
- **Híbridos búsqueda–bioinspirados**: GA memético, ACO+LS y B&B exacto con cota inicial de ACO+LS.

9 Configuración experimental y reproducibilidad

- **Hardware**: Intel Xeon @ 2.10 GHz, 2 vCPU, 7 GB de RAM, Linux. Python 3.13.18, NumPy 2.5.3, SciPy 1.18.1 (Dijkstra, HiGHS), scikit-learn 1.9.1.
- **Tiempo**: `time.perf_counter`, sin instrumentar. **Memoria**: pico de asignaciones de Python con `tracemalloc` en una corrida aparte (no incluye el intérprete).
- **Semillas**: los métodos estocásticos (SA, Tabú, GA, ACO, UCB-HH, híbridos, GA de la mochila, min-conflicts) se corrieron con 10 semillas. Las tablas reportan media ± desviación estándar; las figuras con escala logarítmica, mediana y rango intercuartil. Los métodos deterministas se promedian sobre instancias o consultas distintas.
- **Ejecución**: `python run_all.py` corre todos los experimentos y regenera `resultados/*.csv`, las figuras, las tablas de este reporte (`reporte/tablas`), `reporte/numeros.tex` y el panel. Cada número del texto viene de esos archivos.

Referencias

[1] S. Russell y P. Norvig. *Artificial Intelligence: A Modern Approach*, 4.ª ed. Pearson, 2021.

[2] R. E. Korf. Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence* 27(1):97–109, 1985.

[3] U. K. Sarkar, P. P. Chakrabarti, S. Ghose y S. C. De Sarkar. Reducing reexpansions in iterative-deepening search by controlling cutoff bounds. *Artificial Intelligence* 50(2):207–221, 1991.

[4] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[5] E. Horowitz y S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM* 21(2):277–292, 1974.

[6] A. K. Mackworth. Consistency in networks of relations. *Artificial Intelligence* 8(1):99–118, 1977.

[7] M. Held y R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of SIAM* 10(1):196–210, 1962.

[8] T. Stützle y H. H. Hoos. MAX–MIN Ant System. *Future Generation Computer Systems* 16(8):889–914, 2000.

[9] T. Stützle y M. Dorigo. A short convergence proof for a class of ant colony optimization algorithms. *IEEE Trans. Evolutionary Computation* 6(4):358–365, 2002.

[10] B. Hajek. Cooling schedules for optimal annealing. *Mathematics of Operations Research* 13(2):311–329, 1988.

[11] G. Rudolph. Convergence analysis of canonical genetic algorithms. *IEEE Trans. Neural Networks* 5(1):96–101, 1994.

[12] P. Toth y D. Vigo. The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing* 15(4):333–346, 2003.

[13] P. Auer, N. Cesa-Bianchi y P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning* 47:235–256, 2002.

[14] G. B. Dantzig, D. R. Fulkerson y S. M. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research* 2(4):393–410, 1954.

[15] D. Caffagni. 2D path planning with CNN (dataset y generador), `https://github.com/dcaffo98/path-planning-cnn`, consultado el 3 de noviembre de 2026.